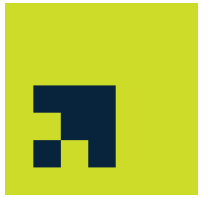
# F107 – Foundations of Computer Science

## Lecture 4

Dr. Sanjit Kumar Saha

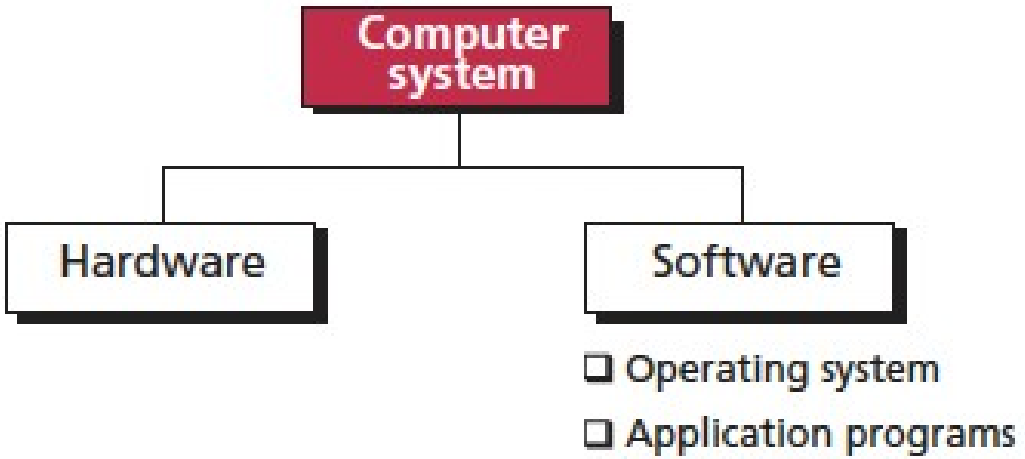
# Topics



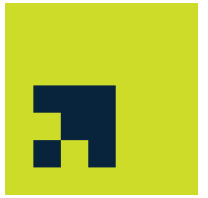
- Operating Systems
  - Types of Software
  - Tasks of an Operating System
    - Mandatory
    - Optional
  - Categories of Operating Systems
  - Computer Start-up Process
  - Examples

# Computer System

- Major components:
  - Hardware: physical equipment
  - Software: collection of programs that allows the hardware to do its job
- Computer software:
  - operating system: controls the access to hardware by users
  - application programs: use the computer hardware to solve users' problems

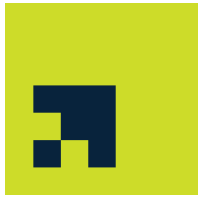


# Operating System



- An operating system is an interface between the hardware of a computer and the user (programs or humans)
  - facilitates the execution of other programs
  - the access to hardware and software resources
- Two major design goals:
  - Efficient use of hardware
  - Easy use of resources

# Operating System vs User Applications

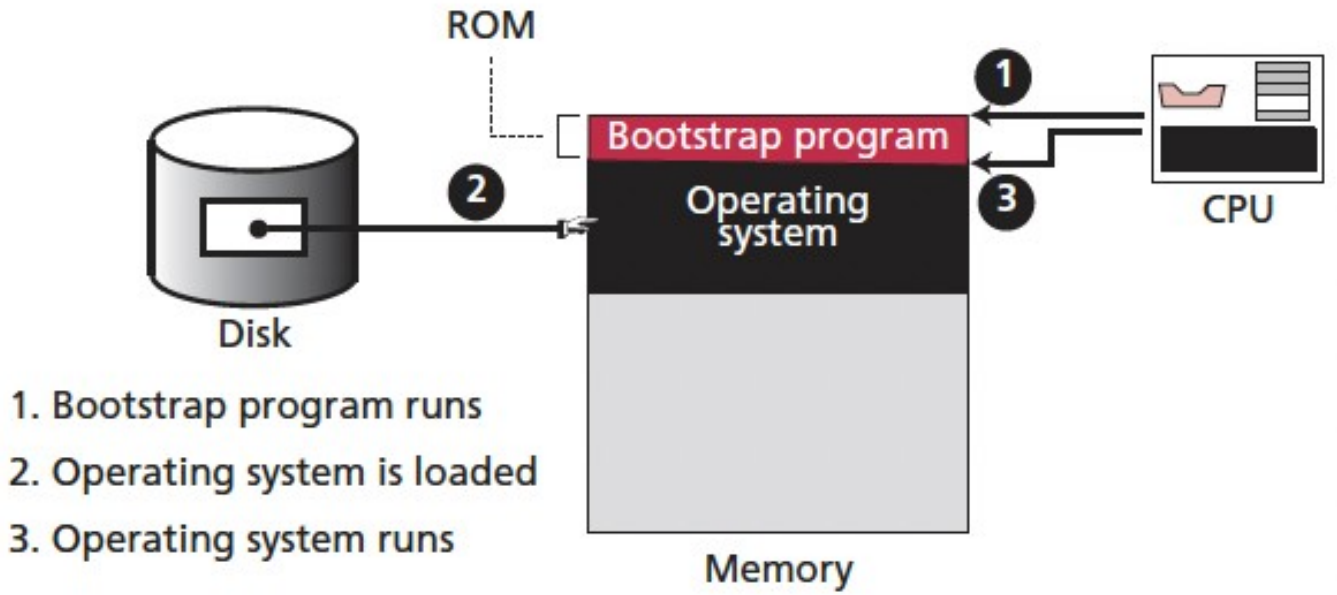


- Operating System
  - Executed with highest permission levels
  - Written in low-level programming languages
  - Very hardware-specific realization, high degree of optimization
  - Independent of any other software components

- User Applications
  - Performs one specific task at the discretion of the user
  - Very interactive with multi-media content
  - Most often GUI-driven
  - Easy to use, install, uninstall, update
  - May be realized in a distributed fashion

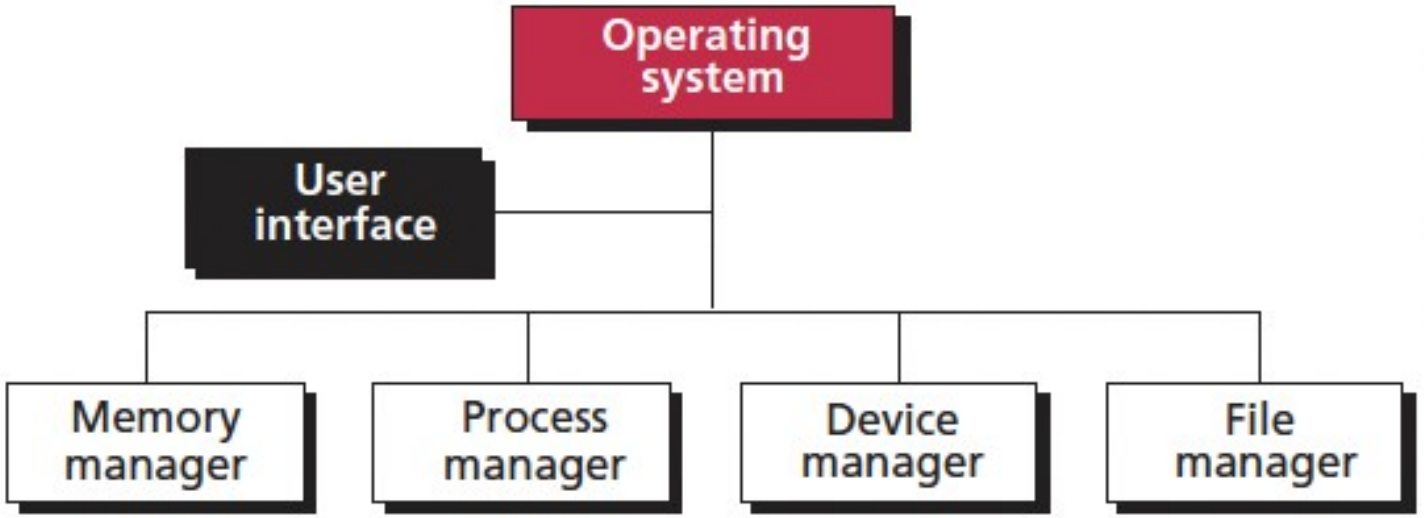
# Bootstrap process

- The operating system itself is a program that needs to be loaded into the memory and be run.



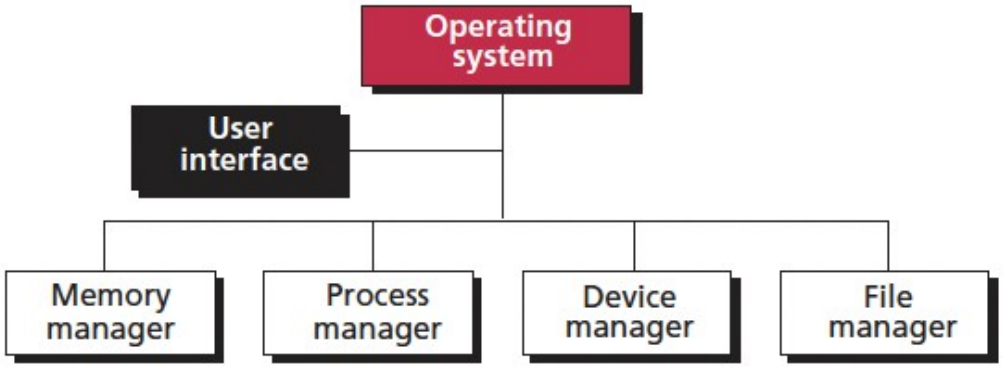
# Components of an Operating System

- An operating system needs to manage different resources in a computer system.
- It resembles an organization with several managers at the top level.
- Each manager is responsible for managing their department, but also needs to cooperate with others and coordinate activities.

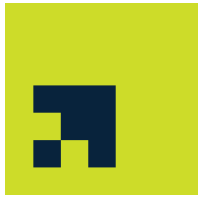


# User Interface

- Each operating system has a user interface,
  - a program that accepts requests from users (processes) and interprets them for the rest of the operating system.
- A user interface in some operating systems, such as UNIX, is called a shell.
- In others, it is called a window to denote that it is menu driven and has a GUI (graphical user interface) component.



# Memory Manager

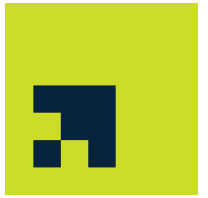


- Memory allocation must be managed to prevent applications from running out of memory.
- Two broad categories:
  - monoprogramming
  - multiprogramming

# Monoprogramming

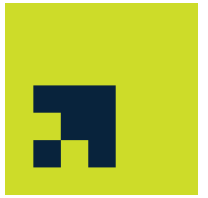
- Most of the memory capacity is dedicated to a single program
  - the data to be processed by a program as part of the program
- only a small part is needed to hold the operating system
- The whole program is in memory for execution.
- When the program finishes running, the program area is occupied by another program

# Monoprogramming...

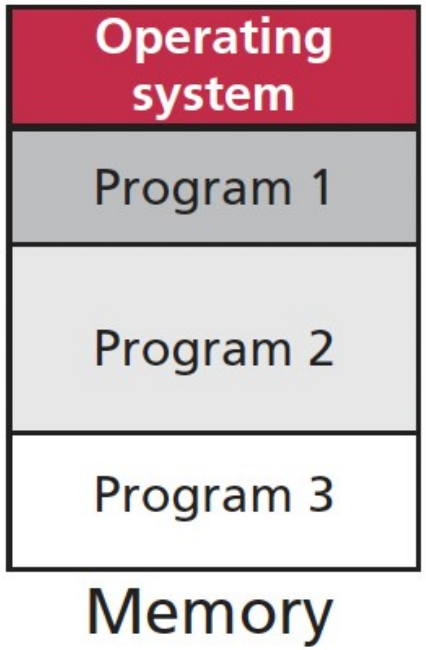


- The memory manager loads the program into memory, runs it, and replaces it with the next program.
- Problems with this technique:
  - The program must fit into memory. If the size of memory is less than the size of the program, the program cannot be run.
  - When one program is being run, no other program can be executed. A program, during its execution, often needs to receive data from input devices and needs to send data to output devices. Input/output devices are slow compared with the CPU, so when the input/output operations are being carried out, the CPU is idle. It cannot serve another program because this program is not in memory. This is a very inefficient use of memory and CPU time.

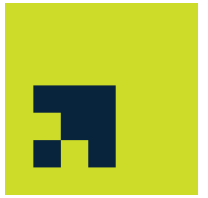
# Multiprogramming



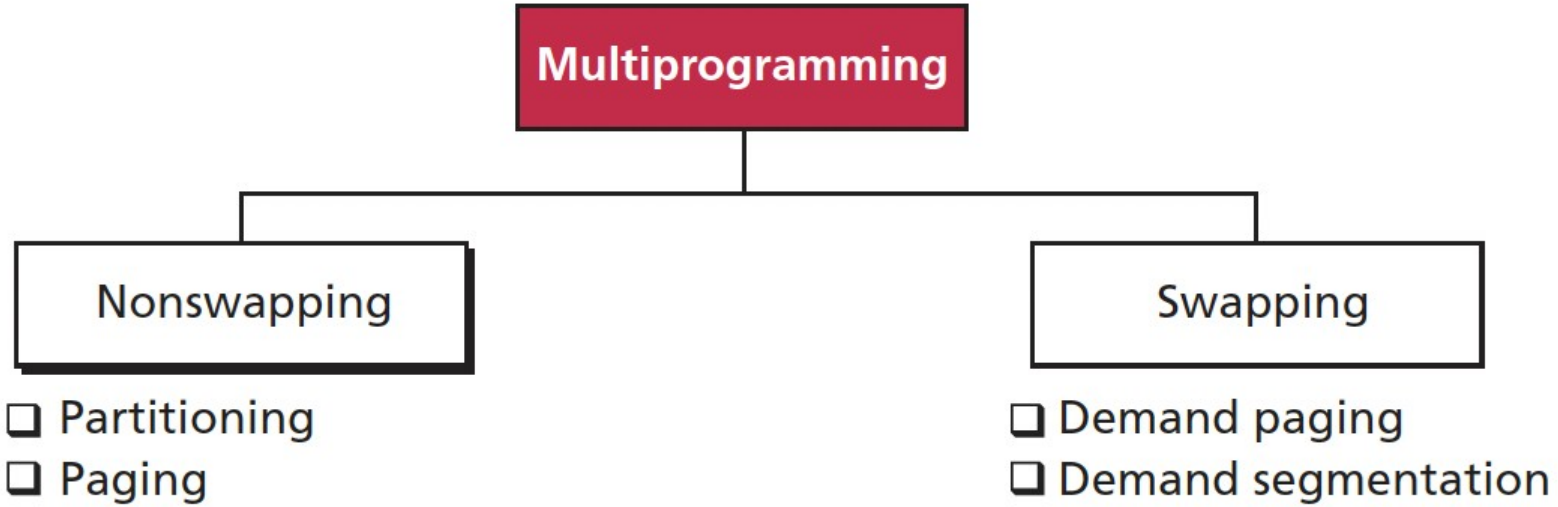
- More than one program is in memory at the same time.
- They are executed concurrently, with the CPU switching rapidly between the programs.



# Categories of multiprogramming

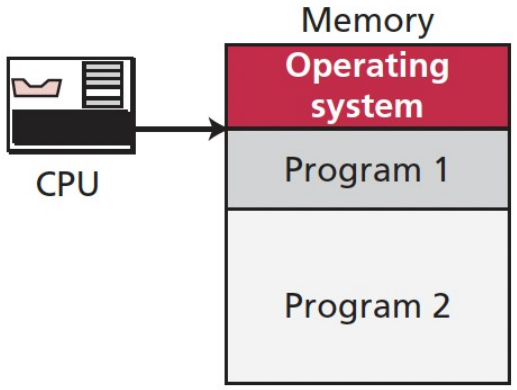


- Nonswapping category
  - the program remains in memory for the duration of execution.
- Swapping category
  - during execution, the program can be swapped between memory and disk one or more times.

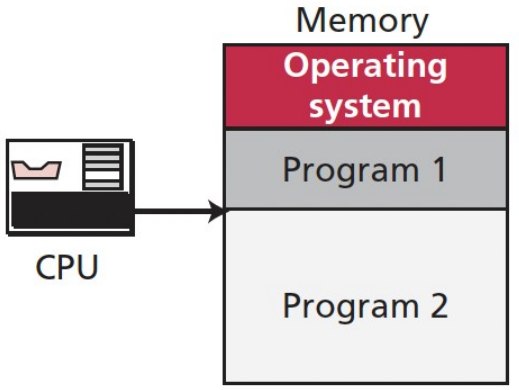


# Partitioning

- Memory is divided into variable-length sections. Each section or partition holds one program.
- The CPU switches between programs.
  - It starts with one program, executing some instructions until it either encounters an input/output operation or the time allocated for that program has expired.
  - The CPU then saves the address of the memory location where the last instruction was executed and moves to the next program.
  - The same procedure is repeated with the second program.
  - After all the programs have been served, the CPU moves back to the first program. Priority levels can also be used to control the amount of CPU time allocated to each program.

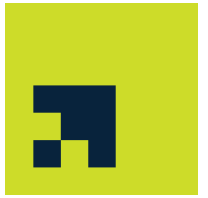



a. CPU starts executing program 1



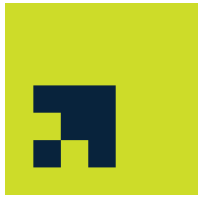

b. CPU starts executing program 2

# Partitioning...



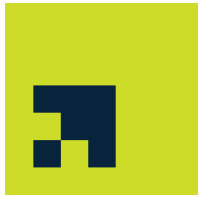
- Partitioning improves the efficiency of the CPU, but there are still some issues:
  - The size of the partitions has to be determined beforehand by the memory manager. If partition sizes are small, some programs cannot be loaded into memory. If partition sizes are large, there might be some ‘holes’ (unused locations) in memory.
  - Even if partitioning is perfect when the computer is started, there may be some holes after completed programs are replaced by new ones.
  - When there are many holes, the memory manager can compact the partitions to remove the holes and create new partitions, but this creates extra overhead on the system.

# Paging



- Paging improves the efficiency of partitioning.
  - memory is divided into equally sized sections called ***frames***.
  - programs are also divided, into equally sized sections called ***pages***.
  - the size of a page and a frame is usually the same and equal to the size of the block used by the system to retrieve information from a storage device

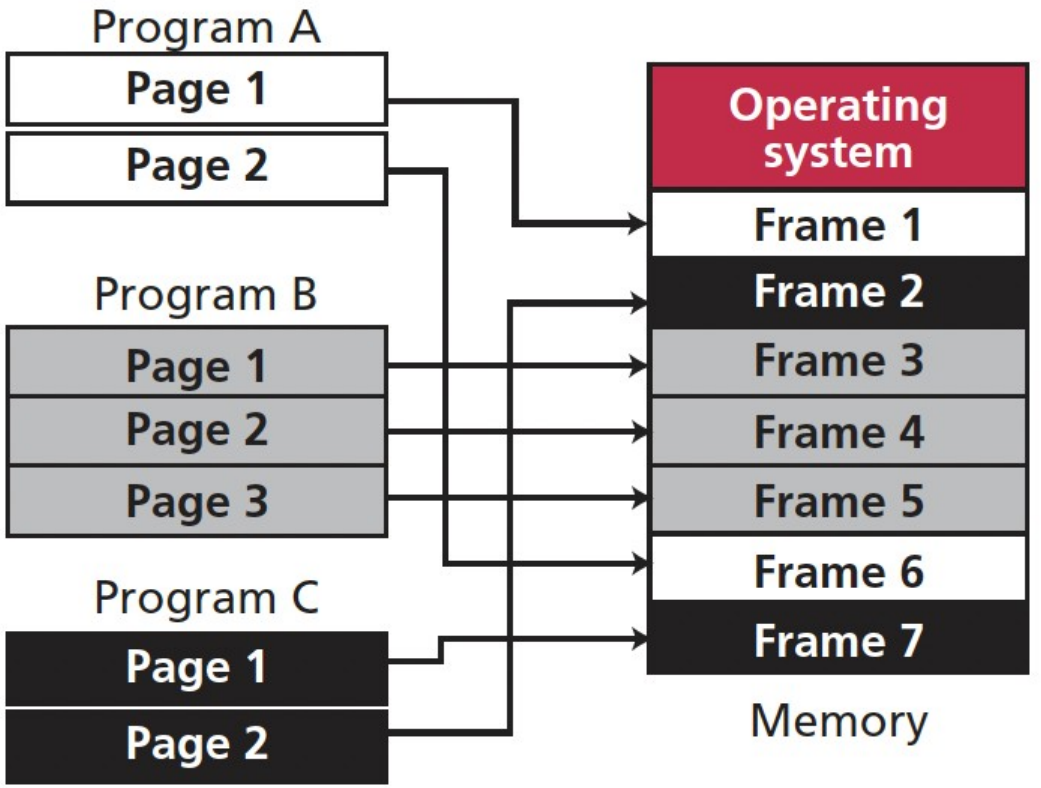
# Paging



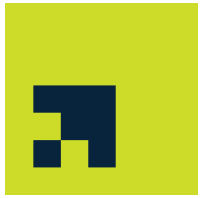
- Paging improves the efficiency of partitioning.
  - memory is divided into equally sized sections called ***frames***.
  - programs are also divided, into equally sized sections called ***pages***.
  - the size of a page and a frame is usually the same and equal to the size of the block used by the system to retrieve information from a storage device

# Paging...

- A page is loaded into a frame in memory.
- If a program has three pages, it occupies three frames in memory.
  - two consecutive pages can occupy noncontiguous frames in memory.
- Advantages:
  - two programs, each using three noncontiguous frames, can be replaced by one program that needs six frames.
  - no need for the new program to wait until six contiguous frames are free before being loaded into memory.

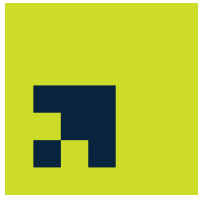


# Paging...



- Limitations:
  - the whole program still needs to be in memory before being executed.
  - a program that needs six frames, for example, cannot be loaded into memory if there are currently only four unoccupied frames.

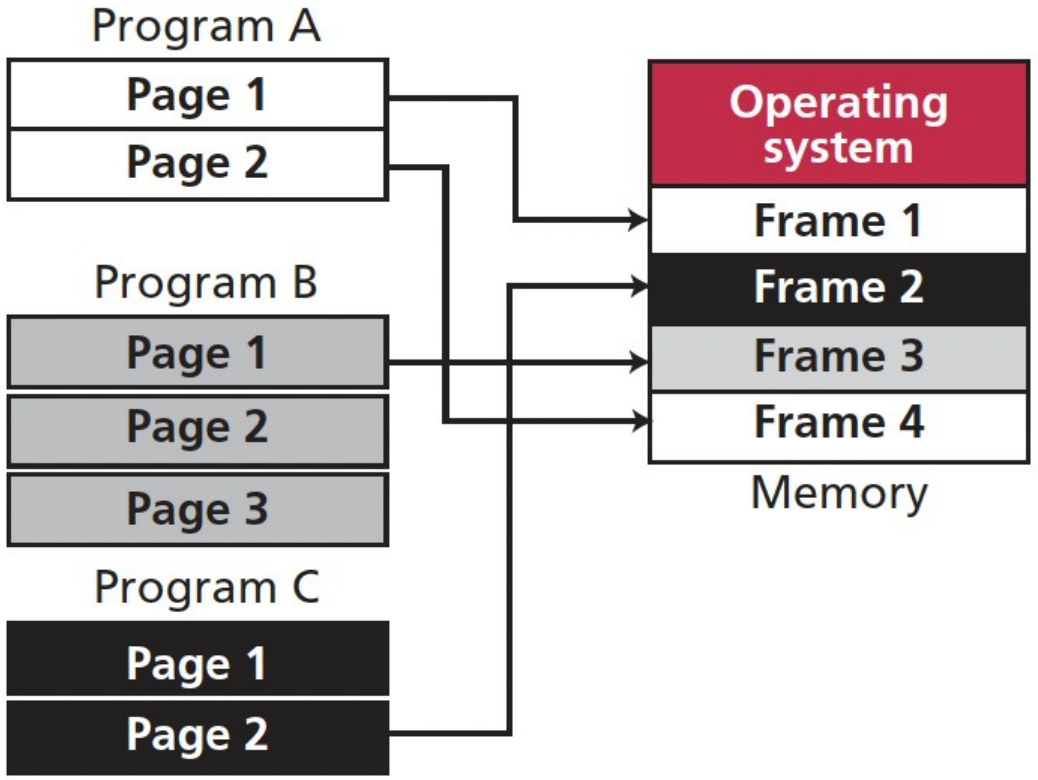
# Demand Paging



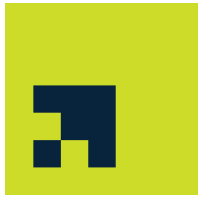
- the program is divided into pages, but the pages can be loaded into memory one by one, executed, and replaced by another page.
- memory can hold pages from multiple programs at the same time.
- consecutive pages from the same program do not have to be loaded into the same frame—a page can be loaded into any free frame.

# Demand Paging...

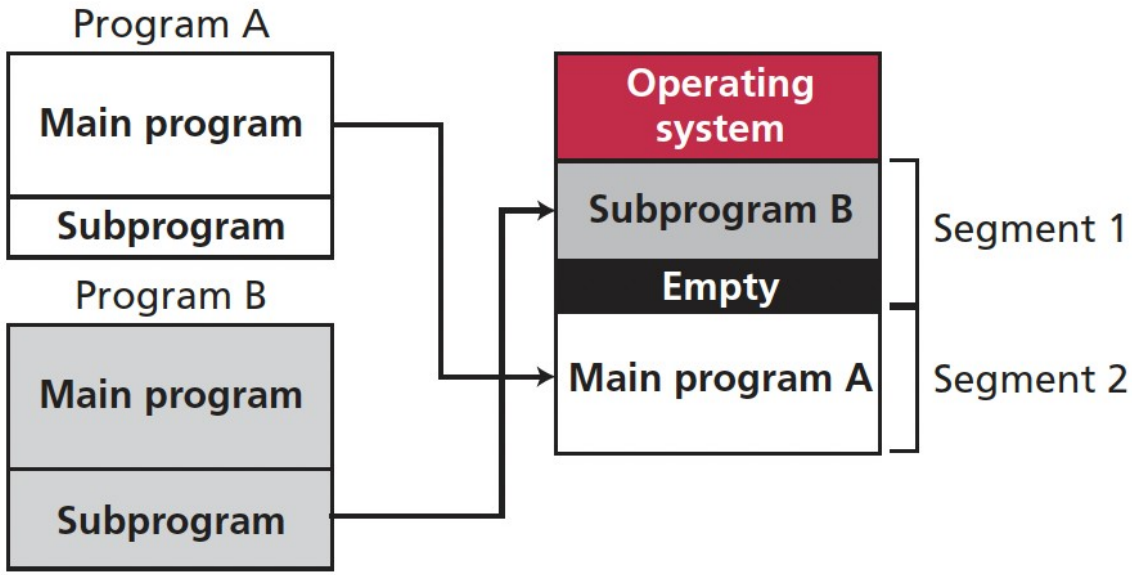
- Two pages from program A, one page from program B, and one page from program C are in the memory.



# Demand Segmentation

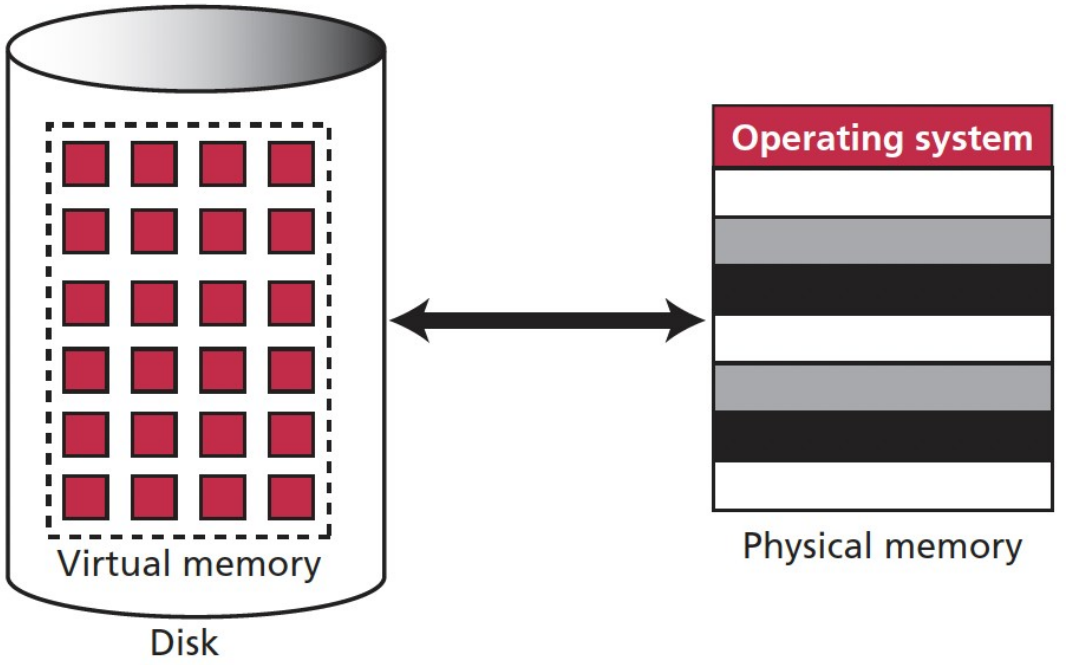


- The program is divided into segments that match the programmer's view.
- These are loaded into memory, executed, and replaced by another module from the same or a different program.
- Since segments in memory are of equal size, part of a segment may remain empty.

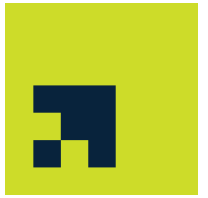


# Virtual memory

- Demand paging and demand segmentation mean that, when a program is being executed, part of the program is in memory and part is on disk.
- For example, a memory size of 10 MB can execute ten programs, each of size 3 MB, for a total of 30 MB.
- At any moment, 10 MB of the ten programs are in memory and 20 MB are on disk.
- There is therefore an actual memory size of 10 MB, but a virtual memory size of 30 MB.

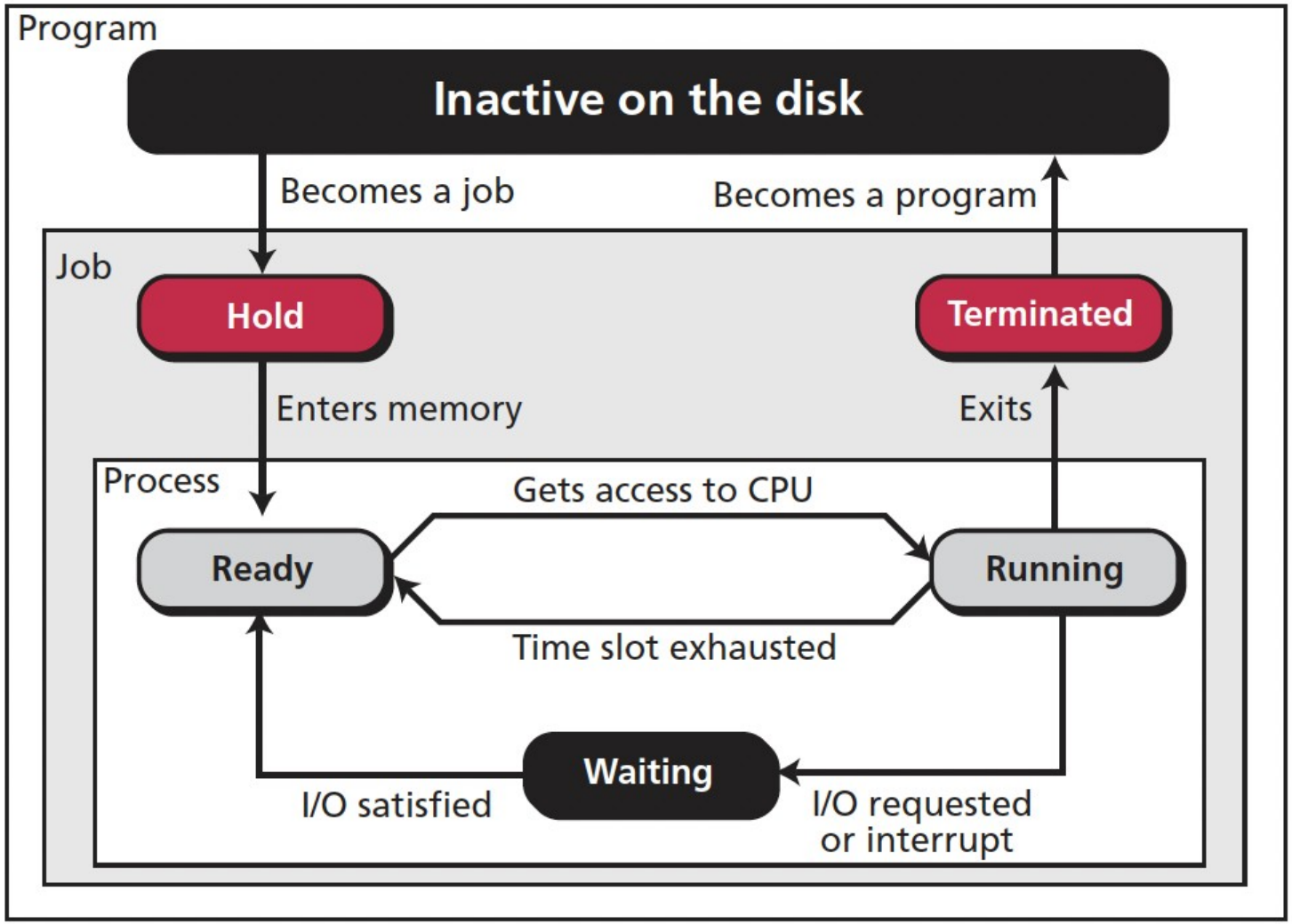


# Program, job, and process

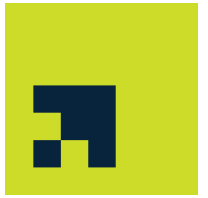


- A **program** is a nonactive set of instructions stored on disk (or tape). It may or may not become a job.
- A program becomes a **job** from the moment it is selected for execution until it has finished running and becomes a program again. During this time a job may or may not be executed.
- A **process** is a program in execution. It is a program that has started but has not finished.

# State diagram



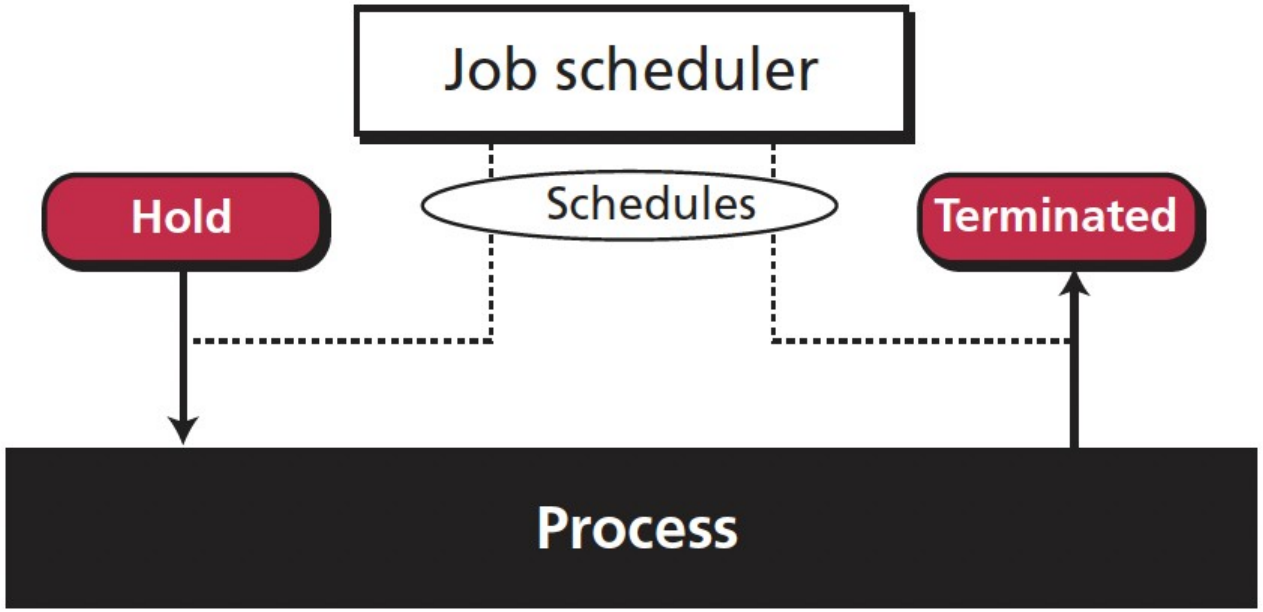
# Schedulers



- To move a job or process from one state to another, the process manager uses two schedulers:
  - the job scheduler
  - the process scheduler

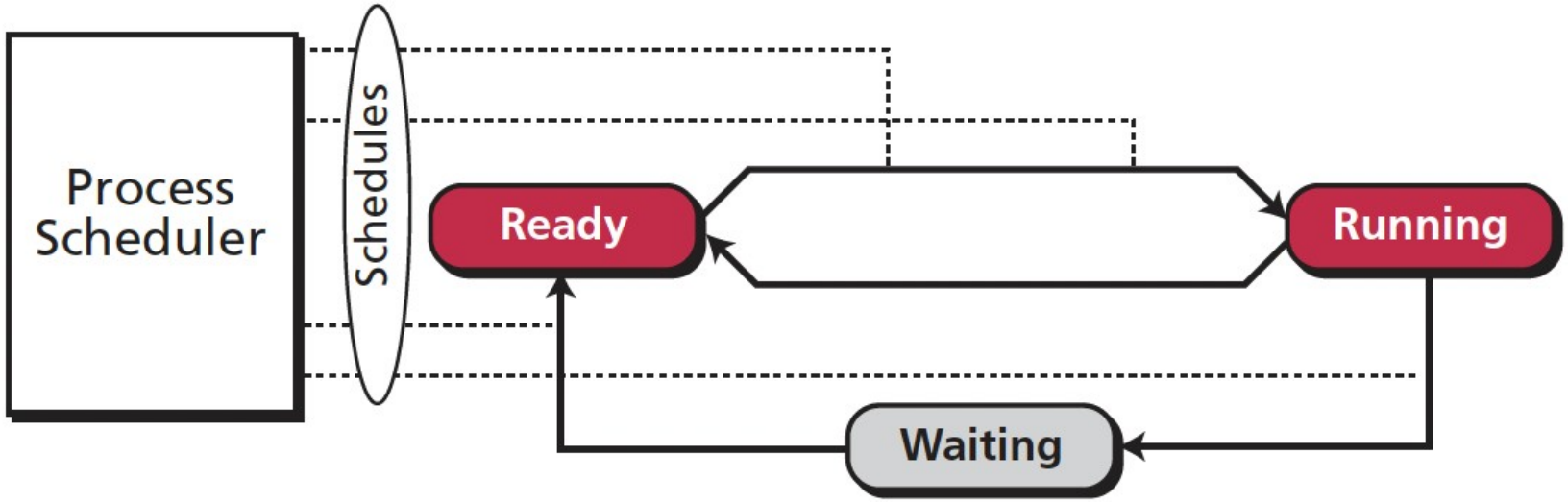
# Job scheduler

- It moves a job from the hold state to the ready state or from the running state to the terminated state.
- It is responsible for creating a process from a job and terminating a process.



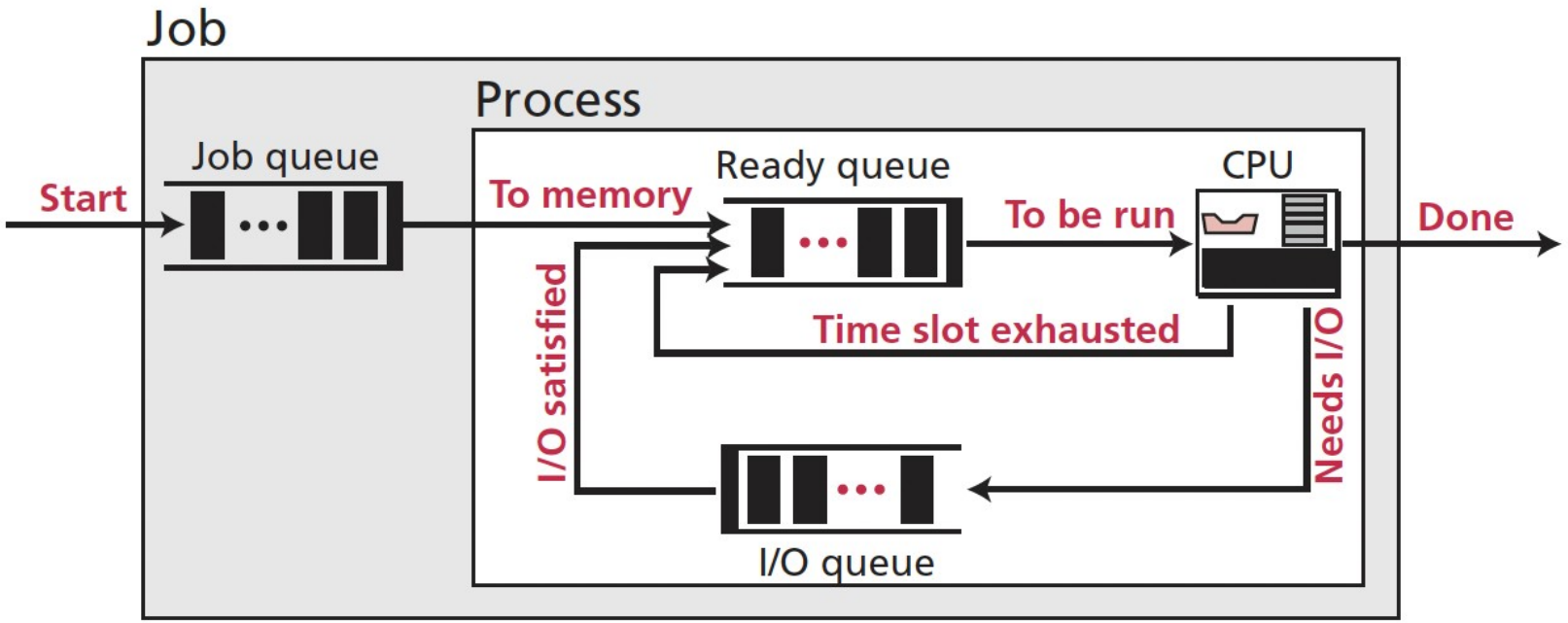
# Process scheduler

- It moves a process from one state to another.
- It moves a process from the running state to the waiting state when the process is waiting for some event to happen.
- It moves the process from the waiting state to the ready state when the event has occurred.

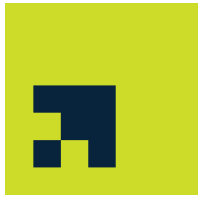


# Queuing

- When some jobs are in memory, others must wait until space is available.
- When a process is running using the CPU, others must wait until the CPU is free.
- To handle multiple processes and jobs, the process manager uses queues (waiting lists).



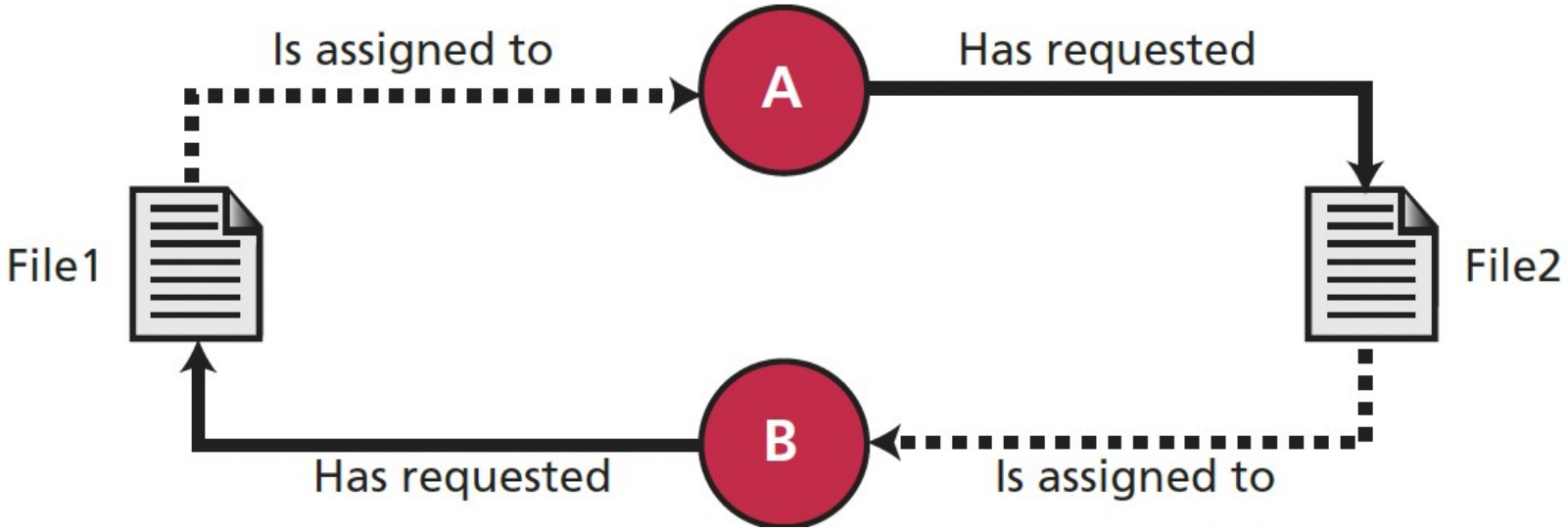
# Process synchronization



- Whenever resources can be used by more than one user (or process), we can have two problematic situations:
  - deadlock
  - starvation

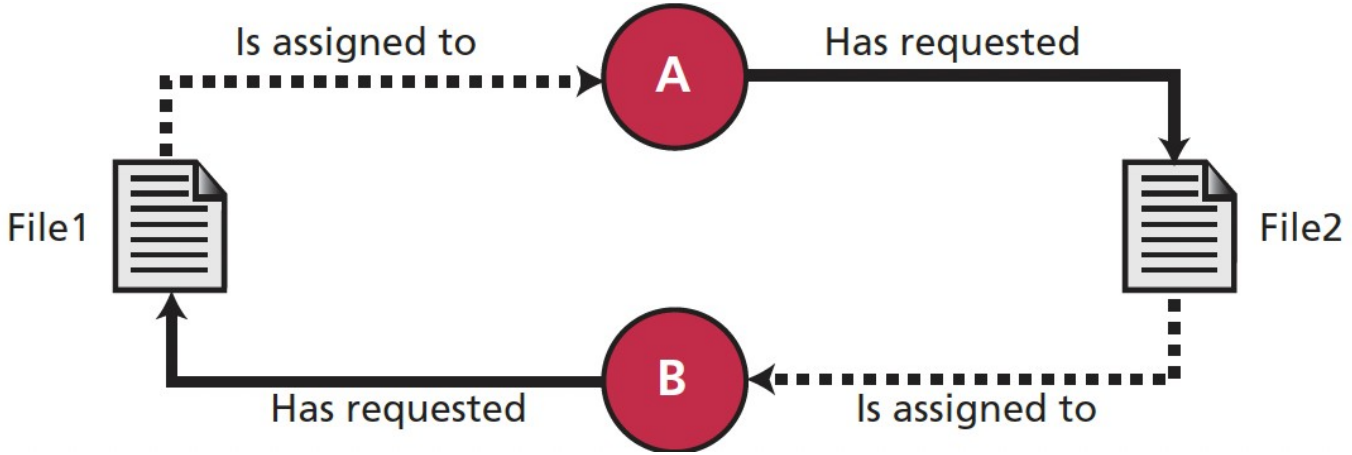
# Deadlock

- Files in most systems are not sharable—when in use by one process, a file cannot be used by another process.



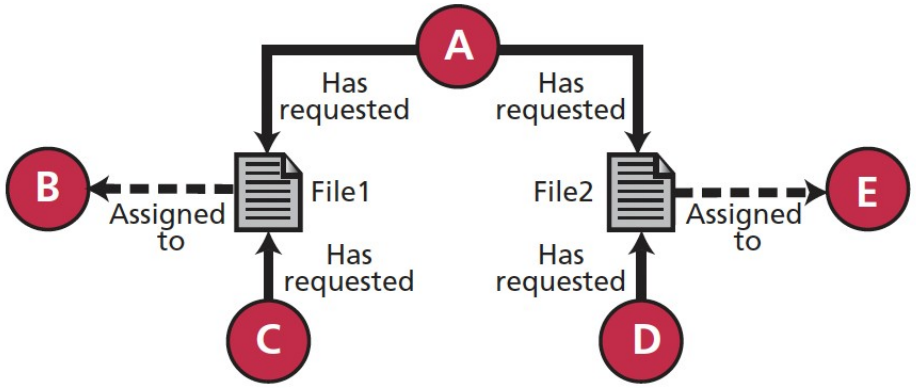
# Deadlock

- Files in most systems are not sharable—when in use by one process, a file cannot be used by another process.
- Deadlock conditions:
  - ***Mutual exclusion:*** Only one process can hold a resource
  - ***Resource holding:*** A process holds a resource even though it cannot use it until other resources are available
  - ***No preemption:*** The operating system cannot temporarily reallocate a resource
  - ***Circular waiting:*** All processes and resources involved form a loop.

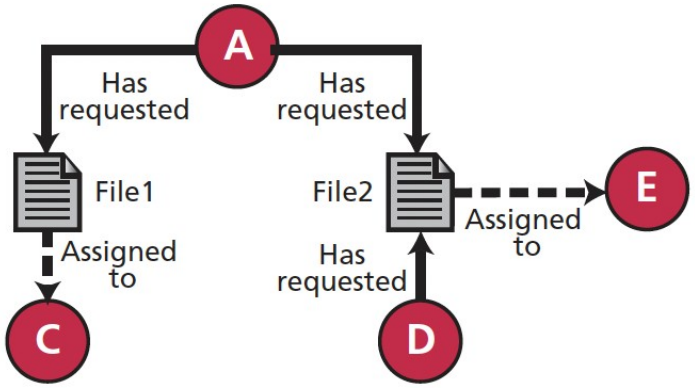


# Starvation

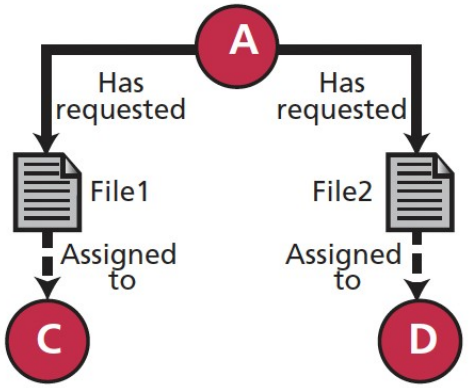
- Starvation can happen when the operating system puts too many resource restrictions on a process.




a. Process A needs both File1 and File2

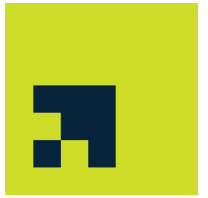



b. Process A still needs both File1 and File2



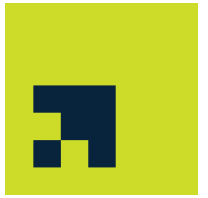

c. Process A still needs both File1 and File2 (starving)

# Device Manager



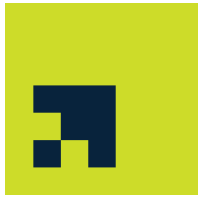
- The device manager, or input/output manager, is responsible for access to input/output devices.
- It monitors every input/output device constantly to ensure that the device is functioning properly.
- It maintains a queue for each input/output device or one or more queues for similar input/output devices.
- It controls the different policies for accessing input/output devices.

# File Manager



- The file manager controls access to files.
- The file manager supervises the creation, deletion, and modification of files.
- The file manager can control the naming of files.
- The file manager supervises the storage of files: how they are stored, where they are stored, and so on.
- The file manager is responsible for archiving and backups.

# Computer Start-up Process



1. Power-On
2. CPU auto-loads **BIOS** (Basic Input/Output System) or **UEFI** (Unified Extensible Firmware Interface) from special memory
   - POST (power-on self-test)
   - Detect primary device*
3. BIOS/UEFI loads **SB** (system bootloader) from primary device's MBR/GPT
4. SB detects primary partition* and load **PB** (partition bootloader)
5. PB loads **OS** kernel into memory
6. OS boots
   - Initialization of kernel
   - Loads further components
7. OS initializes user space

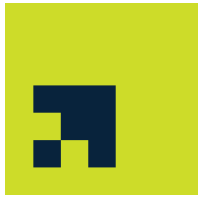
# Task 1

- Define and explain the purpose of an operating system.

# Task 2

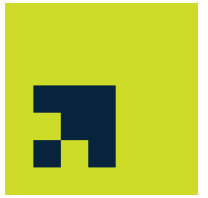
- Describe the components of a typical operating system and their functions.

# Task 3



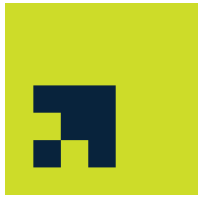
- Explain the process of bootstrapping and how an OS is loaded.

# Task 4



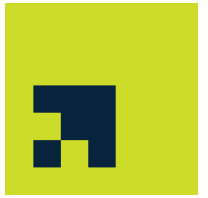
- Compare and contrast monoprogramming and multiprogramming.

# Task 5



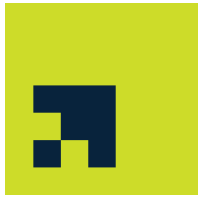
- Design a scheduling algorithm for a real-time OS and justify your choice.

# Task 6



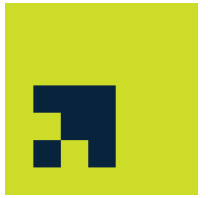
- Explain how memory is managed in a virtual memory system.

# Task 7



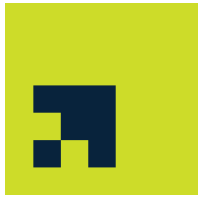
- Create a diagram showing the difference between segmentation and paging.

# Task 8



- Describe deadlock and starvation, and explain how they can be prevented.

# Task 9



- Compare UNIX, Linux, and Windows in terms of structure and usage.